Datasheet C#: formattazione con ToString

Numeri, date, TimeSpan, enum e Guid · esempi con cultura it-IT, .NET 8+

Come funziona

Ogni tipo numerico, `DateTime`, `TimeSpan`, `Guid` ed enum ha `ToString(string format, IFormatProvider provider)`: il **formato** dice come scrivere il valore, il **provider** (di solito una `CultureInfo`) decide separatori, nomi dei mesi e simbolo di valuta. Senza provider si usa la cultura corrente (in `it-IT`: virgola decimale, punto per le migliaia).

```
using System.Globalization;

int n = 1234;
n.ToString();  // "1234"
n.ToString("N2");  // "1.234,00"   (it-IT)
n.ToString("N2", CultureInfo.InvariantCulture);  // "1,234.00"
$"{n:N2}";  // come n.ToString("N2")
string.Format("{0:N2}", n);  // idem
```

Esistono due famiglie di stringhe di formato:

- **Standard**: una lettera + precisione opzionale (`N2`, `X4`, `d`, `T`). Il numero dopo la lettera cambia significato a seconda della lettera.
- **Personalizzate**: un pattern di simboli (`0.00`, `#,0`, `dd/MM/yyyy`) in cui ogni simbolo ha un ruolo e il resto è testo letterale.

Per usare un solo carattere come formato personalizzato di una data (ad esempio solo il giorno) si scrive `%d`; senza `%`, `d` viene letto come formato standard.

Numeri: formati standard

Una lettera sceglie il tipo di formattazione, il numero che segue è la **precisione**: per `F`, `N`, `P`, `C`, `E` sono i decimali; per `D`, `X`, `B` sono le cifre minime (riempie con zeri a sinistra); per `G` sono le cifre significative. Risultati con cultura `it-IT`.

Codice	Significato	Esempio	Risultato
`F` / `F2`	Decimali fissi	`3.14159.ToString("F2")`	`3,14`
`N` / `N2`	Decimali + separatore migliaia	`1234567.891.ToString("N1")`	`1.234.567,9`
`D` / `D5`	Intero con zeri iniziali (solo interi)	`42.ToString("D5")`	`00042`
`X` / `x` / `X4`	Esadecimale maiuscolo/minuscolo (solo interi)	`255.ToString("X4")`	`00FF`
`B` / `B8`	Binario (solo interi, .NET 8+)	`5.ToString("B8")`	`00000101`
`E` / `E2`	Scientifico	`1234.5.ToString("E2")`	`1,23E+003`
`P` / `P1`	Percentuale (moltiplica per 100)	`0.256.ToString("P1")`	`25,6%`
`C` / `C2`	Valuta della cultura	`1234.5.ToString("C2")`	`1.234,50 €`
`G` / `G3`	Generale, forma più compatta	`3.14159.ToString("G3")`	`3,14`

Note: `ToString()` senza argomenti equivale a `G` e, da .NET Core 3.0, dà un valore che rileggendolo restituisce lo stesso `double`. Gli spazi e i simboli esatti di `P` e `C` possono variare leggermente con versione di .NET e sistema operativo.

Numeri: formati personalizzati

Un pattern descrive la forma finale cifra per cifra: `0` forza la cifra, `#` la scrive solo se significativa. Tutto ciò che non è un simbolo speciale viene copiato così com'è.

Simbolo	Funzione
`0`	Cifra obbligatoria: se manca, scrive `0`
`#`	Cifra opzionale: se sarebbe uno zero non significativo, non scrive nulla
`.`	Posizione del separatore decimale
`,`	Tra due cifre: separatore delle migliaia. Subito prima del `.` o in fondo: divide per 1000 per ogni virgola
`%` / `‰`	Moltiplica per 100 / per 1000 e scrive il simbolo
`E+0` / `e-00`	Notazione scientifica
`\`	Il carattere successivo è letterale
`'testo'` o `"testo"`	Testo letterale
`;`	Separa fino a 3 sezioni: positivo; negativo; zero

```
1234.5678.ToString("0.00");  // "1234,57"
1234.5.ToString("#.##");  // "1234,5"      (niente zeri inutili)
5.ToString("000");  // "005"
1234567.ToString("#,0");  // "1.234.567"
1234567.ToString("#,0,");  // "1.235"       (diviso 1000)
0.256.ToString("0.0%");  // "25,6%"
1234.5.ToString("0.00 'EUR'");  // "1234,50 EUR"
42.ToString("ID-0000");  // "ID-0042"
(-5).ToString("+0;-0;zero");  // "-5"   (0 → "zero", 5 → "+5")
```

Arrotondamento: la formattazione arrotonda il caso a metà **lontano da zero** (`2.5.ToString("F0")` dà 3), mentre `Math.Round(2.5)` arrotonda al pari più vicino (2). Formattare non tronca mai: per troncare usa `Math.Truncate` prima.

Interi: esadecimale, binario, cifre fisse

Per l'esadecimale basta `X`; per il binario e l'ottale si usa `Convert.ToString(valore, base)` (basi ammesse: 2, 8, 10, 16), oppure `B` da .NET 8. Le cifre fisse si ottengono con la precisione (`X4`, `D6`) o con `PadLeft`.

```
int n = 202;

// Esadecimale
n.ToString("X");  // "CA"
n.ToString("x4");  // "00ca"
$"0x{n:X4}";  // "0x00CA"
Convert.ToString(n, 16);  // "ca"

// Binario e ottale
Convert.ToString(n, 2);  // "11001010"
Convert.ToString(n, 2).PadLeft(16, '0'); // "0000000011001010"
n.ToString("B16");  // "0000000011001010"  (.NET 8+)
Convert.ToString(n, 8);  // "312"

// Cifre fisse e larghezza
n.ToString("D6");  // "000202"
$"{n,8}|";  // "     202|"  (larghezza 8, a destra)
$"{n,-8}|";  // "202     |"  (a sinistra)
$"{n,8:D6}|";  // "  000202|"

// Byte come stringa esadecimale
Convert.ToHexString(new byte[] { 1, 171, 255 }); // "01ABFF"  (.NET 5+)

// Percorso inverso
Convert.ToInt32("CA", 16);  // 202
Convert.ToInt32("11001010", 2);  // 202
int.Parse("CA", NumberStyles.HexNumber);
```

Con i numeri negativi `X` e `Convert.ToString(n, 2)` mostrano il complemento a due sulla larghezza del tipo: `(-1).ToString("X")` dà `FFFFFFFF` per `int` e `FF` per `sbyte`. `D`, `X` e `B` valgono solo per gli interi: su un `double` lanciano `FormatException`.

Date: formati standard

Una sola lettera dà un formato già pronto della cultura. Per solo data usa `d`/`D`, per solo ora `t`/`T`, per data e ora `g`/`G`/`f`/`F`. Esempio con `new DateTime(2026, 10, 19, 22, 23, 45)` in `it-IT`; `ToString()` senza argomenti equivale a `G`.

Codice	Cosa mostra	Risultato
`d`	Data breve	`19/10/2026`
`D`	Data estesa	`lunedì 19 ottobre 2026`
`t`	Ora breve	`22:23`
`T`	Ora completa	`22:23:45`
`g`	Data breve + ora breve	`19/10/2026 22:23`
`G`	Data breve + ora completa	`19/10/2026 22:23:45`
`f`	Data estesa + ora breve	`lunedì 19 ottobre 2026 22:23`
`F`	Data estesa + ora completa	`lunedì 19 ottobre 2026 22:23:45`
`M` / `m`	Giorno e mese	`19 ottobre`
`Y` / `y`	Mese e anno	`ottobre 2026`
`s`	ISO 8601 ordinabile, senza fuso	`2026-10-19T22:23:45`
`o`	ISO 8601 round-trip, con frazioni e fuso	`2026-10-19T22:23:45.0000000`
`u`	Ordinabile universale (non converte in UTC)	`2026-10-19 22:23:45Z`
`U`	Estesa, convertita in UTC	`lunedì 19 ottobre 2026 20:23:45`
`R` / `r`	RFC 1123 (HTTP)	`Mon, 19 Oct 2026 22:23:45 GMT`

`s`, `o`, `u`, `R` non dipendono dalla cultura: sono quelli giusti per log, file, API e database. Gli altri seguono la cultura e vanno bene per la visualizzazione.

Solo data o solo ora da un `DateTime`: `d.ToString("d")`, `d.ToString("T")`, oppure `d.Date` e `d.TimeOfDay` se ti servono come valori.

Date: formati personalizzati

Un formato personalizzato è una sequenza di simboli: la **ripetizione** di una lettera decide la lunghezza (`d` = 19 o 5, `dd` = 19 o 05). Attenzione a maiuscole e minuscole: `MM` è il mese, `mm` i minuti; `HH` è l'ora 0-23, `hh` l'ora 1-12.

Esempio con `19/10/2026 22:23:45.123`.

Simbolo	Significato	Risultato
`d` / `dd`	Giorno del mese, senza / con zero	`19`
`ddd`	Giorno della settimana abbreviato	`lun`
`dddd`	Giorno della settimana esteso	`lunedì`
`M` / `MM`	Mese numerico, senza / con zero	`10`
`MMM`	Mese abbreviato	`ott`
`MMMM`	Mese esteso	`ottobre`
`yy` / `yyyy`	Anno a 2 / 4 cifre	`26` / `2026`
`H` / `HH`	Ora 0-23, senza / con zero	`22`
`h` / `hh`	Ora 1-12, senza / con zero	`10`
`m` / `mm`	Minuti	`23`
`s` / `ss`	Secondi	`45`
`f`, `ff`, `fff`...	Frazioni di secondo (zeri mostrati)	`fff` → `123`
`F`, `FF`, `FFF`...	Frazioni di secondo (zeri finali omessi)	`FFF` → `123`
`tt`	Indicatore AM/PM	`PM`
`zzz`	Scarto dal fuso orario	`+02:00`
`K`	Fuso (`Z` per UTC, scarto per l'ora locale)	`+02:00`
`/` e `:`	Separatori di data e ora **della cultura**	`/` e `:`
`'testo'` o `\`	Testo letterale	

```
var d = new DateTime(2026, 10, 19, 22, 23, 45, 123);

d.ToString("ddMM");  // "1910"
d.ToString("ddMMyyyy_HHmmss");  // "19102026_222345"   (adatto ai nomi file)
d.ToString("yyyy-MM-dd HH:mm:ss.fff");  // "2026-10-19 22:23:45.123"
d.ToString("dddd d MMMM yyyy");  // "lunedì 19 ottobre 2026"
```

```
d.ToString("dd/MM/yy");  // "19/10/26"
d.ToString("HH 'ore' mm 'minuti'");  // "22 ore 23 minuti"
d.ToString("hh:mm tt", CultureInfo.InvariantCulture); // "10:23 PM"
d.ToString("%d");  // "19"  (singolo simbolo: serve il %)
```

Tre regole pratiche:

- Le lettere che non vuoi interpretare vanno tra apici: `"dd/MM/yyyy 'alle' HH:mm"`. Senza apici, `a` e `t` verrebbero letti come simboli.
- `/` e `:` sono sostituiti dai separatori della cultura: in `it-IT` restano uguali, ma con altre culture possono diventare `.` o `-`. Per forzarli scrivi `"dd'/'MM'/'yyyy"` o usa `InvariantCulture`.
- Nei nomi dei file non usare `:`; usa `HHmmss`.

Culture, interpolazione e allineamento

Lo stesso formato dà risultati diversi a seconda della cultura. Regola semplice: per **mostrare** all'utente usa la cultura corrente; per **scrivere su file, JSON, log, database o inviare in rete** usa sempre `CultureInfo.InvariantCulture`, altrimenti un `1234.5` diventa `1234,5` e il programma che lo rilegge si rompe.

```
var it  = new CultureInfo("it-IT");
var us  = new CultureInfo("en-US");
var inv = CultureInfo.InvariantCulture;

1234.5.ToString("N2", it);  // "1.234,50"
1234.5.ToString("N2", us);  // "1,234.50"
1234.5.ToString("F2", inv);  // "1234.50"
DateTime.Now.ToString("D", us); // "Monday, October 19, 2026"

// Cambiare la cultura di tutto il programma
CultureInfo.DefaultThreadCurrentCulture = it;
```

In una stringa il formato si scrive `{valore,larghezza:formato}`: la larghezza positiva allinea a destra, negativa a sinistra. Vale sia per `string.Format` (con indici `{0}`) sia per l'interpolazione `$"..."`.

```
string nome = "Penna"; double prezzo = 1.5; DateTime d = DateTime.Now;

string.Format("{0,-10}{1,8:N2}", nome, prezzo);  // "Penna          1,50"
$"{nome,-10}{prezzo,8:N2}";  // identico
$"{d:dd/MM/yyyy HH:mm}";  // formato dentro le graffe
$"{{letterali}} {prezzo:C2}";  // "{letterali} 1,50 €"
$"{(prezzo > 1 ? "caro" : "ok")}";  // il ? : vuole le parentesi

// Interpolazione con cultura precisa
FormattableString.Invariant($"{prezzo:F2}");  // "1.50"
```

In `it-IT` l'indicatore `tt` (AM/PM) può risultare vuoto: per averlo sempre usa `InvariantCulture` o `en-US`.

TimeSpan, enum, Guid, DateOnly e TimeOnly

`TimeSpan` ha regole sue: nei formati personalizzati `:` e `.` **vanno protetti** con `\` o apici, e non esistono `HH`, `MM`, `yyyy`. I simboli sono `d`, `h`, `m`, `s`, `f` e rappresentano la **componente**, non il totale: per il totale usa `TotalHours`, `TotalMinutes`...

```
var ts = new TimeSpan(1, 2, 3, 4, 500);  // 1 giorno, 2 h, 3 min, 4 s, 500 ms

ts.ToString();  // "1.02:03:04.5000000"  (formato c)
ts.ToString("g");  // "1:2:03:04,5"         (breve, cultura)
ts.ToString(@"hh\:mm\:ss");  // "02:03:04"
ts.ToString(@"d\.hh\:mm");  // "1.02:03"
ts.ToString(@"mm\:ss\.fff");  // "03:04.500"
ts.TotalHours.ToString("F1");  // "26,1"   (ore totali, non solo la componente)
```

Enum, Guid, DateOnly, TimeOnly:

Tipo	Formato	Effetto	Esempio
enum	`G` (default)	Nome	`Colore.Verde.ToString()` → `Verde`
enum	`D`	Valore numerico	`Colore.Verde.ToString("D")` → `2`
enum	`X`	Valore in esadecimale a 8 cifre	`Colore.Verde.ToString("X")` → `00000002`
enum `[Flags]`	`G`	Nomi separati da virgola	`(A | B).ToString()` → `A, B`
`Guid`	`D` (default)	Con trattini	`3f2504e0-4f89-11d3-9a0c-0305e82c3301`
`Guid`	`N`	32 cifre senza trattini	`3f2504e04f8911d39a0c0305e82c3301`
`Guid`	`B` / `P`	Tra graffe / tonde	`{3f2504e0-...}` / `(3f2504e0-...)`
`DateOnly` (.NET 6+)	come `DateTime`	Solo formati di data	`DateOnly.FromDateTime(d).ToString("dd/MM/yyyy")`
`TimeOnly` (.NET 6+)	come `DateTime`	Solo formati di ora	`TimeOnly.FromDateTime(d).ToString("HH:mm")`
`DateTimeOffset`	come `DateTime`	In più `zzz` e `K` per il fuso	`DateTimeOffset.Now.ToString("o")`

Trappole comuni e ricette pronte

Gli errori più frequenti nascono da maiuscole e minuscole, dalla cultura e dai caratteri non protetti.

Errore	Cosa succede	Correzione
`"dd/mm/yyyy"`	`mm` sono i minuti: `19/23/2026`	`"dd/MM/yyyy"`
`"hh:mm"` per le 22:23	hh è 1-12: `10:23`	`"HH:mm"`
`x.ToString()` scritto su CSV/JSON	Con `it-IT` esce la virgola decimale	`x.ToString(CultureInfo.InvariantCulture)`
`"dd/MM/yyyy at HH:mm"`	`a` e `t` interpretati come simboli	`"dd/MM/yyyy 'at' HH:mm"`
`d.ToString("d")` voluto come giorno	È il formato standard data breve	`d.ToString("%d")`
`3.5.ToString("X")`	`FormatException`: `X`, `D`, `B` solo per interi	`((int)3.5).ToString("X")`
`int? x; x.ToString("N2")`	Non compila: `Nullable<T>` non ha il formato	`x?.ToString("N2")` oppure `$"{x:N2}"`
`ts.ToString("hh:mm")`	Errore: `:` non protetto	`ts.ToString(@"hh\:mm")`
`Math.Round(2.5)` vs `2.5.ToString("F0")`	`2` contro `3` (arrotondamenti diversi)	Scegli quello voluto e restane coerente

Ricette:

Scopo	Codice
Nome file con timestamp	`DateTime.Now.ToString("yyyyMMdd_HHmmss")`
Data per log, JSON, API (UTC)	`DateTime.UtcNow.ToString("o")`
Data italiana	`d.ToString("dd/MM/yyyy")`
Prezzo in euro	`prezzo.ToString("C2", new CultureInfo("it-IT"))`
Percentuale	`rapporto.ToString("P1")`
Codice con zeri iniziali	`id.ToString("D5")` oppure `$"TKT-{id:00000}"`
Byte in esadecimale	`b.ToString("X2")`
Byte in binario a 8 bit	`Convert.ToString(b, 2).PadLeft(8, '0')`
Durata `hh:mm:ss`	`ts.ToString(@"hh\:mm\:ss")`
Colonne allineate	`$"{nome,-15}{prezzo,10:N2}"`

Direzione inversa: ogni formato di data ha il suo parser. `ParseExact` richiede lo stesso pattern usato per scrivere, `TryParseExact` non lancia eccezioni.

```
DateTime d = DateTime.ParseExact("19102026", "ddMMyyyy", CultureInfo.InvariantCulture);

if (DateTime.TryParseExact("19/10/2026 22:23", "dd/MM/yyyy HH:mm",
        CultureInfo.InvariantCulture, DateTimeStyles.None, out var risultato))
{
    Console.WriteLine(risultato);
}

double.TryParse("1234,5", NumberStyles.Float, new CultureInfo("it-IT"), out var x); // 1234.5
```

Documentazione ufficiale (Microsoft Learn): *Standard numeric format strings*, *Custom numeric format strings*, *Standard date and time format strings*, *Custom date and time format strings*, *Standard TimeSpan format strings*, *Custom TimeSpan format strings*.